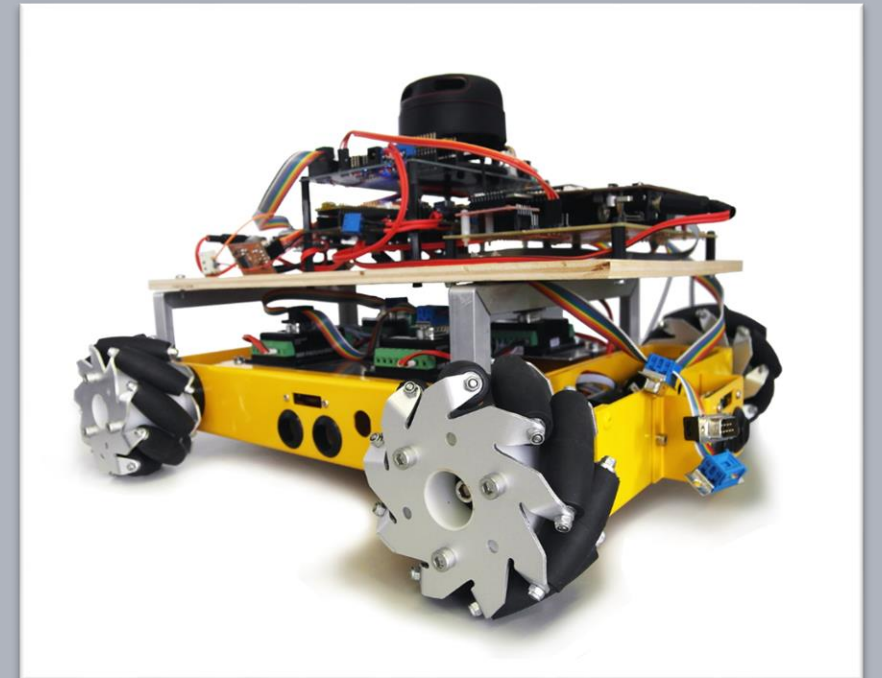


Advanced Embedded Architectures and Applications

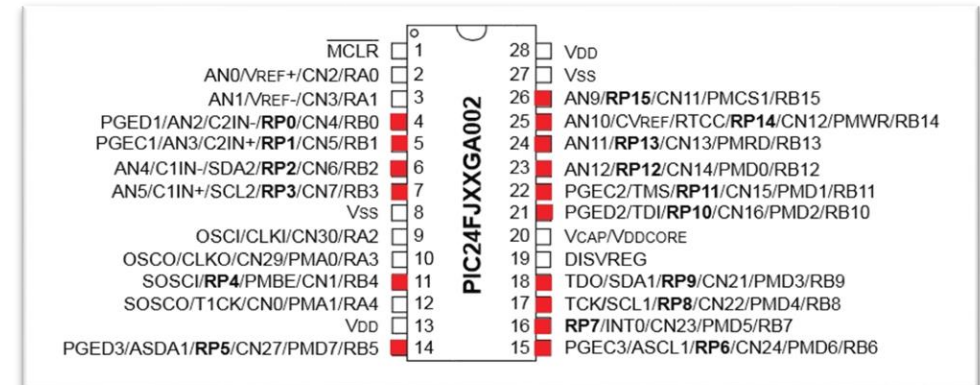
Driver Development

Prof. Dr.-Ing. Peter Fromm



Content

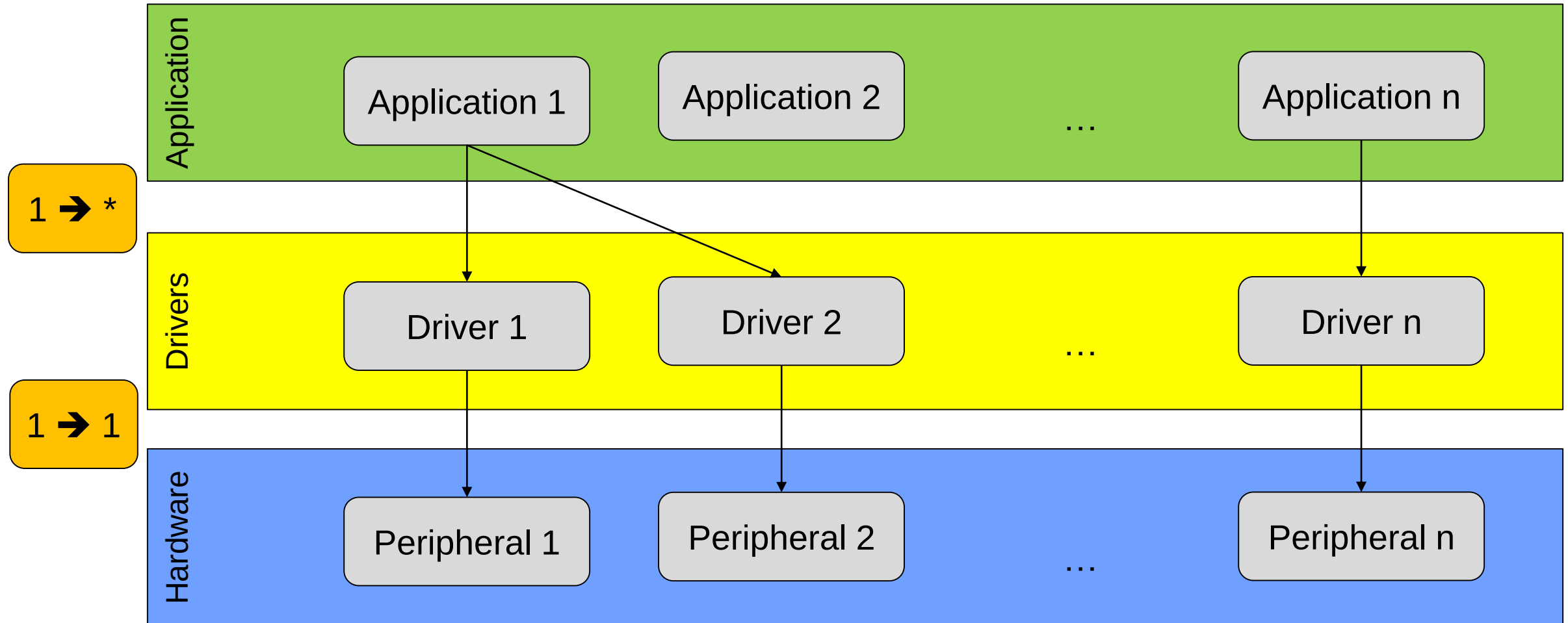
- Some general ideas and requirements
- MCU hardware concepts
- Driver layers
- Pin Configuration
- Peripheral API Design
- Synchronous versus Asynchronous
- Interrupt Service Calls
- OS integration / abstraction
- Critical sections
- Complex Device Driver (CDD)
- A first driver for our car



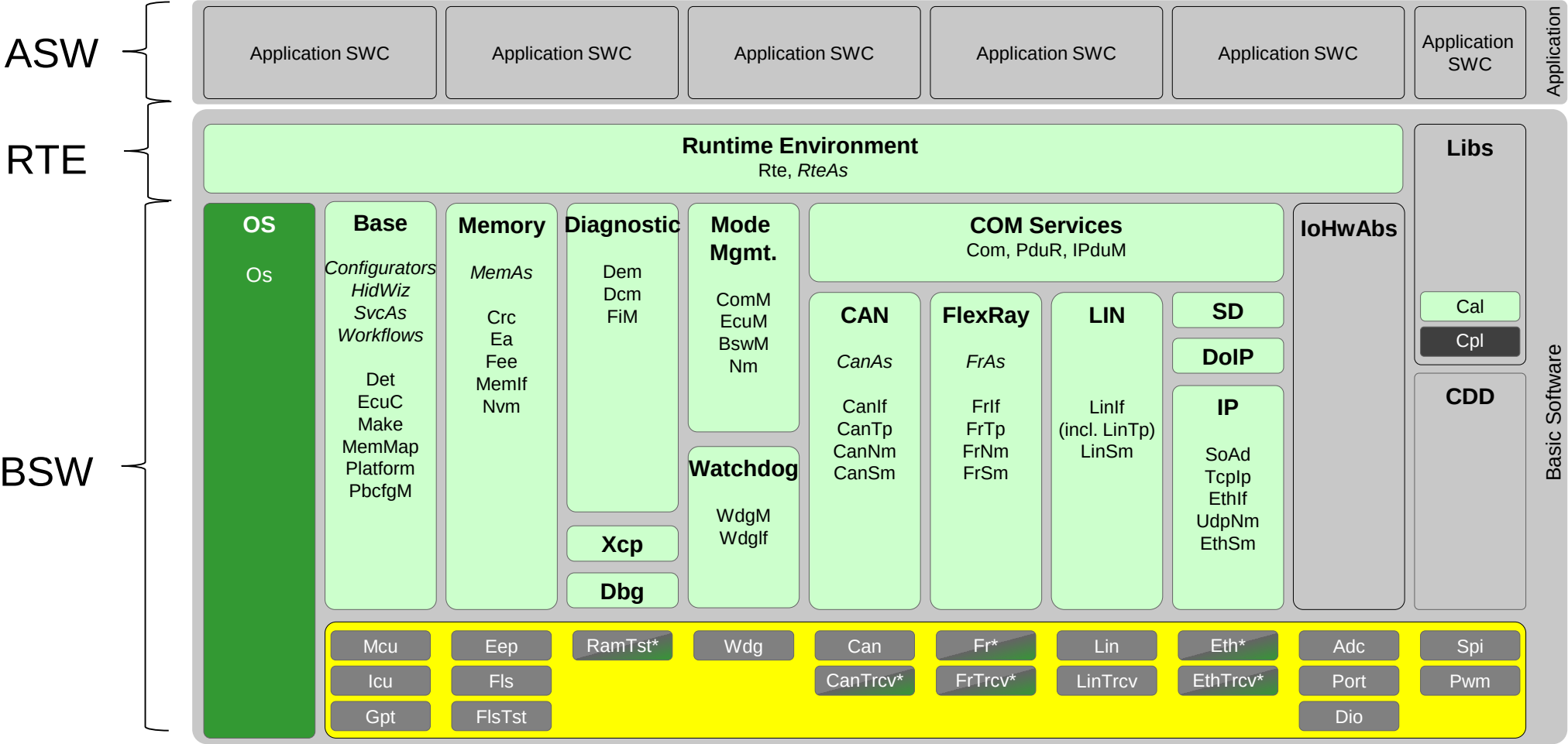
Some general ideas and requirements

- Peripheral drivers, in the context of AUTOSAR also referred to as Microcontroller Abstraction Layer (MCAL) are low level drivers providing direct access to the hardware.
- These drivers are part of the so-called BSW – basic software, as opposite to the ASW – application software.

A very high level perspective



AUTOSAR perspective



Requirements

Functional Requirements

- Configure and initialize the peripheral
- Activate / Deactivate the port
- Read / Write (synchronous or asynchronous)

Non-Functional Requirements

- Good performance
- Take best advantage of hardware features
- Focus on peripheral, not application
- Abstracted and simple to use unique API
- Reentrant / threadsafe
- Loose coupling to OS
- Easily dockable to middleware / runtime environment



Compromises needed!

MCU hardware concepts – MB90460 family (simple, straight forward)

Traditional setup

- A pin can be configured to have GPIO or special function (e.g. USART) functionality
- In addition, the data direction (IN, OUT) of the pin can be set

Figure 3-1 Block Diagram of USART pins

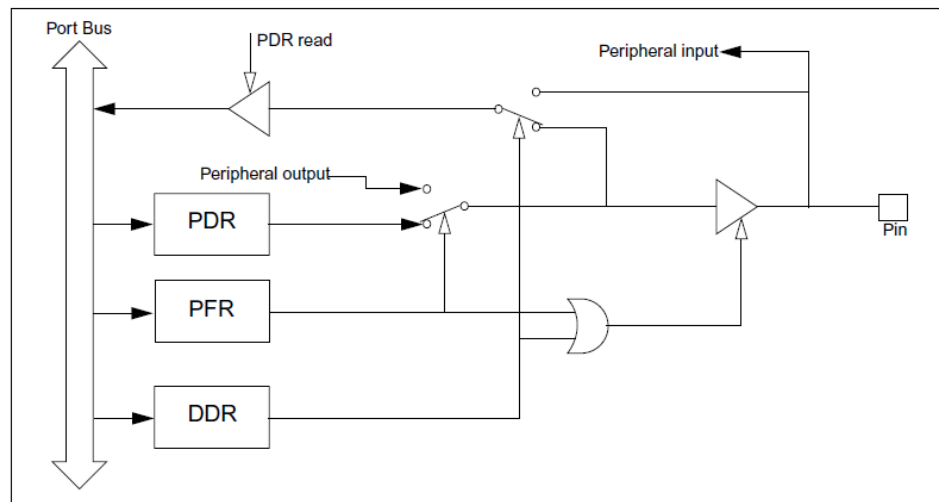
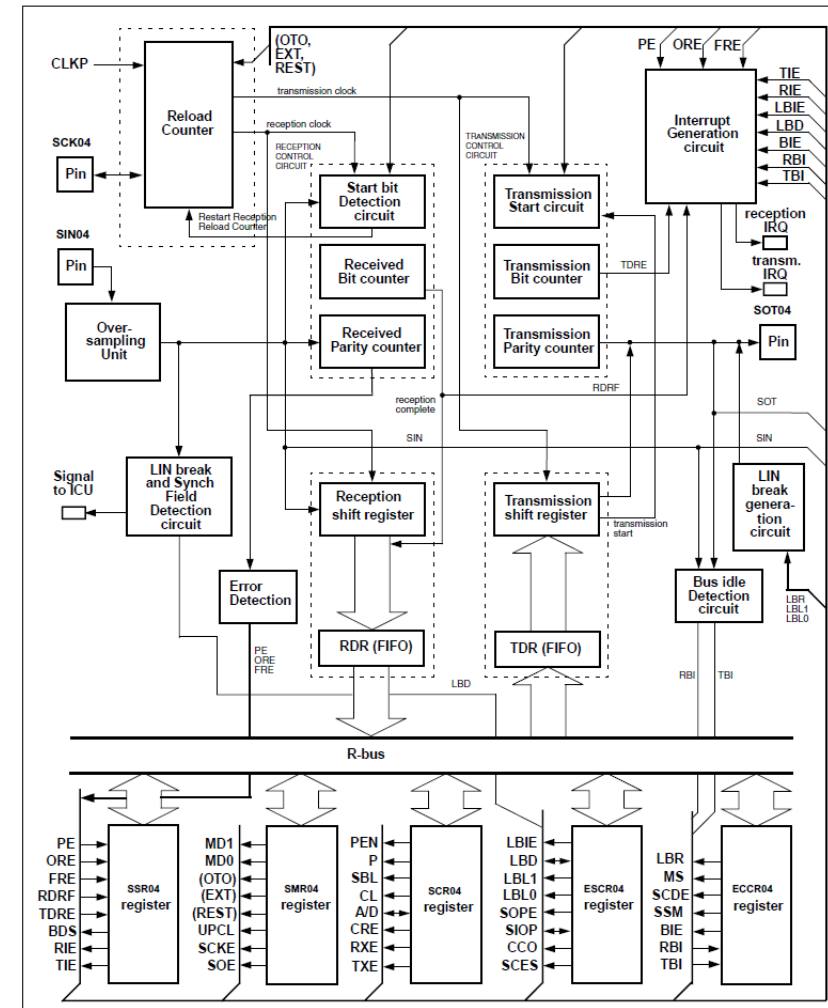


Figure 2-1 USART Block Diagram



MCU Hardware Concepts – Multiplexed Ports (Aurix 297 family)

- Using an additional multiplexer, special functions can be mapped to different pins

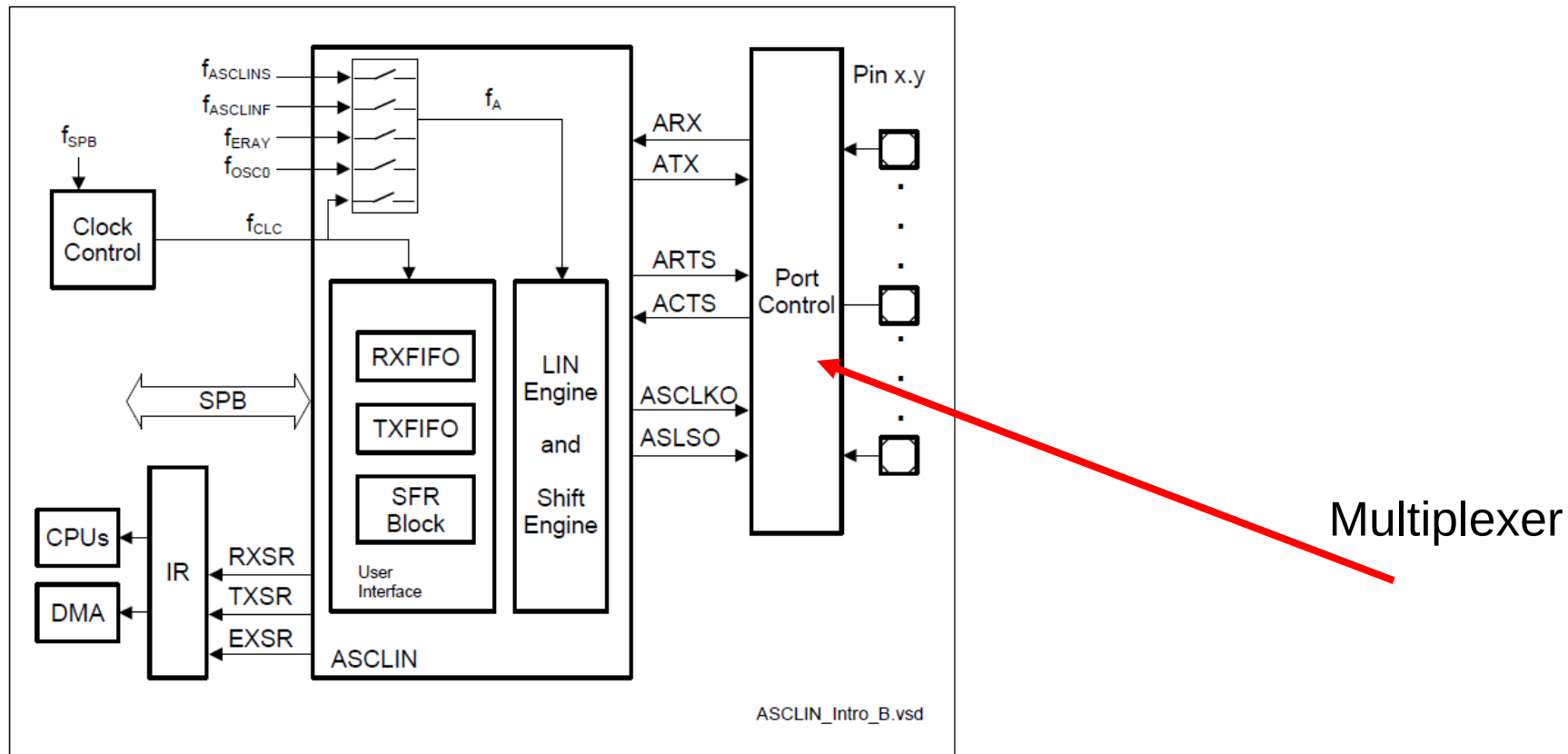
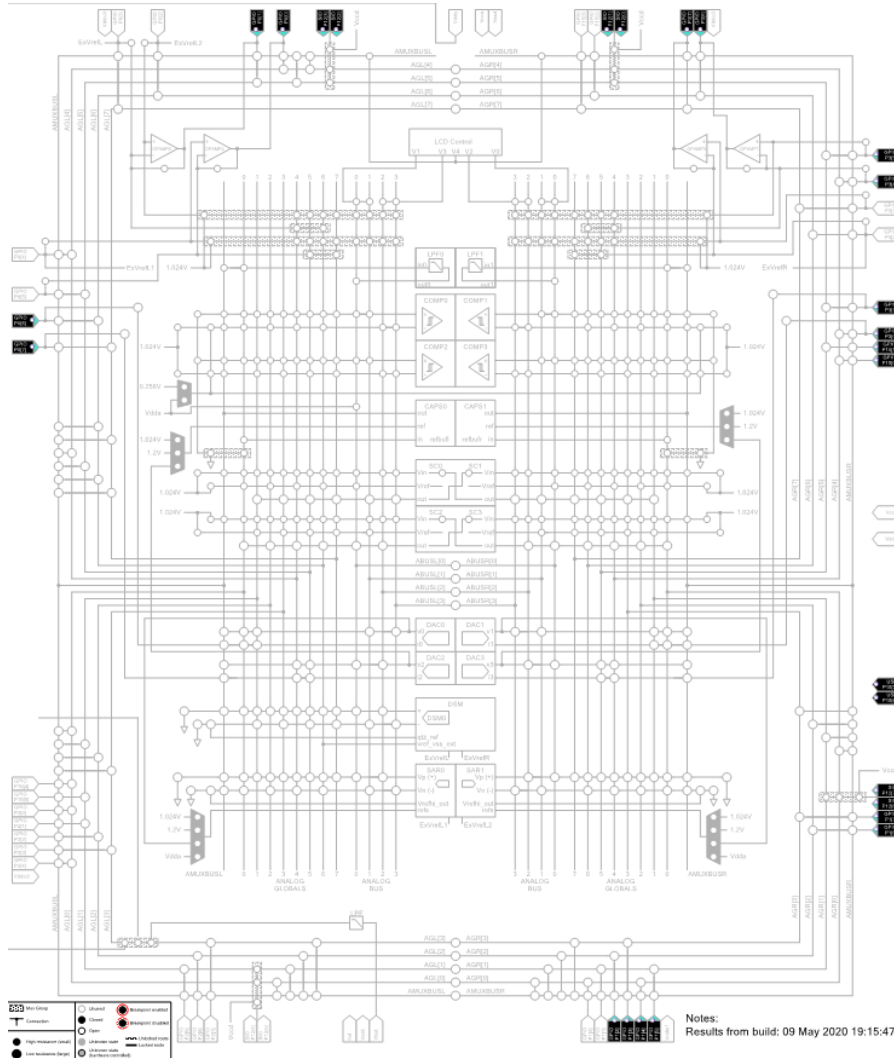


Figure 18-1 Block Diagram of the ASCLIN module.

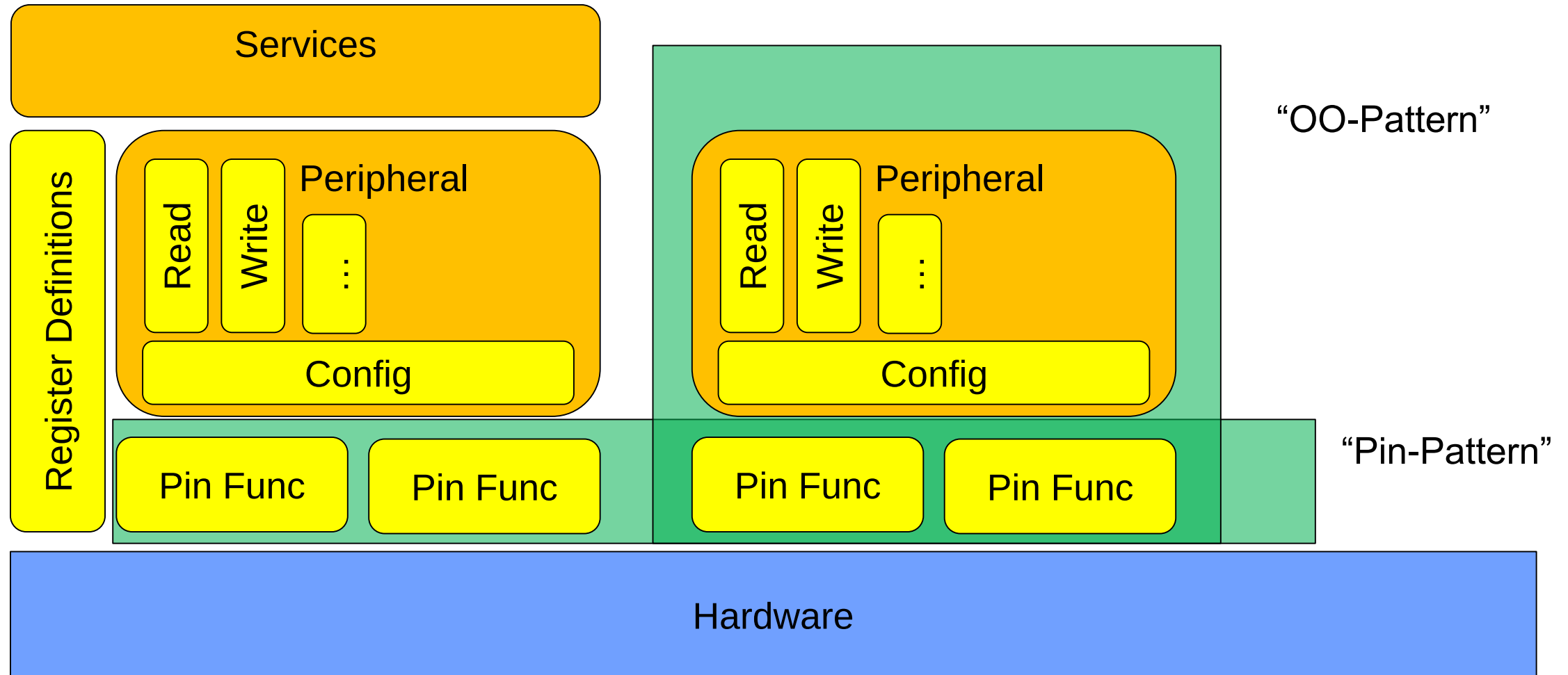
MCU Hardware Concepts – FPGA Universal Design Blocks (PSOC Style)



Trend towards pin flexibility



Driver partitions and layers



Pin Configuration

OO-Pattern

- Focus on the peripheral block as a whole
- Application is configuring peripheral INCLUDING pins

- `ASCLIN_init(PIN_3, PIN_4, 9600Baud,...)`

→ Peripheral & Pin functionality form one unit, which is communicated through some HW pins.

Pin-Pattern

- Focus on having overview on all pins
- Pin configuration is stored in flash
- Option: Peripheral block is configured independently

- `PIN_CONFIG = {
 {PIN_3, ASCLIN_1_tx, 9600Baud}
 {PIN_4, ASCLIN_1_rx, 9600Baud}}`

Alternative

- `PIN_CONFIG = {
 {PIN_3, ASCLIN_1_tx}
 {PIN_4, ASCLIN_1_rx}}`
- `ASCLIN_init(ASCLIN1, 9600, even)`

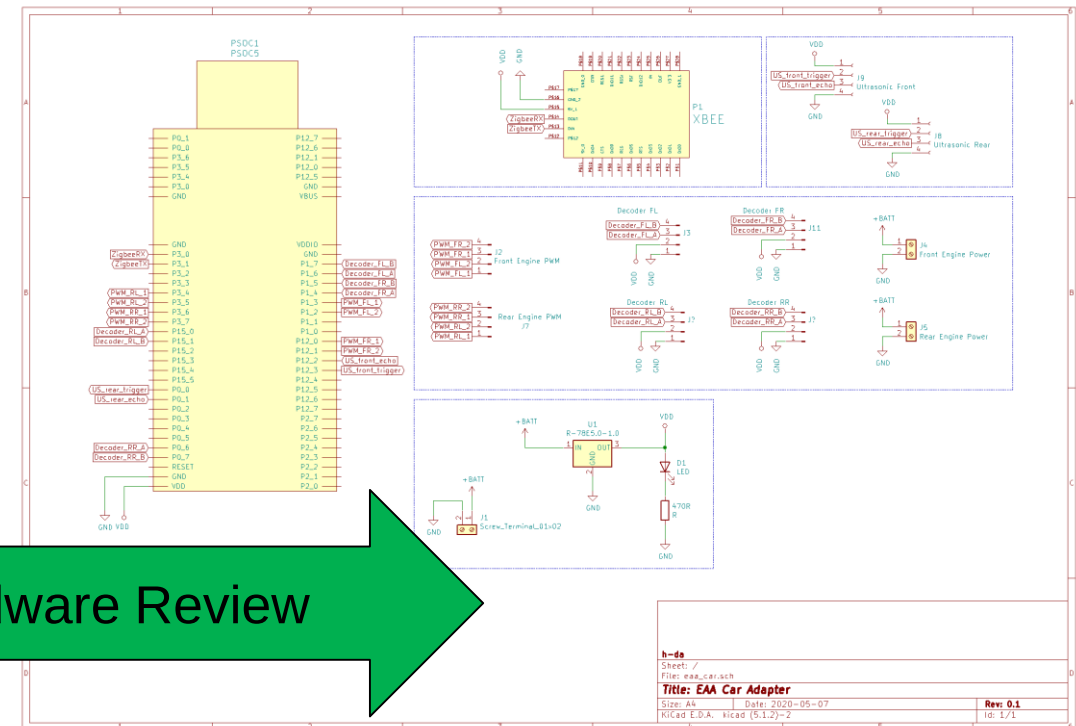
→ At first we define the functionality of all the pins with huge table. On top of this we are placing the peripherals read/write services etc.,

Pin Configuration

Pin-Pattern

Name	Port	Pin	Lock
\USBUART:Dm\	P15[7]	23	✓
\USBUART:Dp\	P15[6]	22	✓
DEBUG_RX	P12[6]	20	✓
DEBUG_TX	P12[7]	21	✓
FL_QDA	P1[6]	18	✓
FL_QDB	P1[7]	19	✓
FR_QDA	P1[4]	15	✓
FR_QDB	P1[5]	16	✓
PWM_FL_1	P1[3]	14	✓
PWM_FL_2	P1[2]	13	✓
PWM_FR_1	P12[0]	38	✓
PWM_FR_2	P12[1]	39	✓
PWM_RL_1	P3[4]	33	✓
PWM_RL_2	P3[5]	34	✓
PWM_RR_1	P3[6]	36	✓
PWM_RR_2	P3[7]	37	✓
RL_QDA	P15[0]	27	✓
RL_QDB	P15[1]	28	✓
RR_QDA	P15[2]	29	✓
RR_QDB	P15[3]	30	✓
US_front_echo	P12[3]	47	✓
US_front_trigger	P0[1]	49	✓
US_rear_echo	P0[0]	48	✓
US_rear_trigger	P0[0]	48	✓
Zigbee_Rx	P3[0]	29	✓
Zigbee_Tx	P3[1]	30	✓

Supports Hardware Review



Pin Configuration

Autosar Pin Concept

The screenshot displays the Autosar Pin Configuration tool interface. On the left, the Project Explorer shows the V3_Port_Dio project structure, including the TMS570LC4357 (ARM, TMS570LCx) component and its associated configuration files. The Outline pane on the left shows the configuration hierarchy, with PortConfigSet_0 selected, and N2HET2 highlighted under the PortContainer.

The main window displays the Port (Port) container configuration for N2HET2. The PortPin table lists the configuration for 19 pins (Index 0 to 18). The table columns include Index, Name, PortPinDirection, PortPinDirection..., PortPinId, PortPinInitialMode, PortPinLevelValue, and Pc.

Index	Name	PortPinDirection	PortPinDirection...	PortPinId	PortPinInitialMode	PortPinLevelValue	Pc
0	PortPinN2Het2_0	PORT_PIN_OUT	☒	0	PORT_PIN_MODE_PWM	PORT_PIN_LEVEL_LOW	POR
1	PortPinN2Het2_1	PORT_PIN_OUT	☒	1	PORT_PIN_MODE_PWM	PORT_PIN_LEVEL_LOW	POR
2	PortPinN2Het2_2	PORT_PIN_IN	☒	2	PORT_PIN_MODE_PWM	PORT_PIN_LEVEL_LOW	POR
3	PortPinN2Het2_3	PORT_PIN_OUT	☒	3	PORT_PIN_MODE_PWM	PORT_PIN_LEVEL_LOW	POR
4	PortPinN2Het2_4	PORT_PIN_IN	☒	4	PORT_PIN_MODE_PWM	PORT_PIN_LEVEL_LOW	POR
5	PortPinN2Het2_5	PORT_PIN_IN	☒	5	PORT_PIN_MODE_PWM	PORT_PIN_LEVEL_LOW	POR
6	PortPinN2Het2_6	PORT_PIN_IN	☒	6	PORT_PIN_MODE_PWM	PORT_PIN_LEVEL_LOW	POR
7	PortPinN2Het2_7	PORT_PIN_OUT	☒	7	PORT_PIN_MODE_PWM	PORT_PIN_LEVEL_LOW	POR
8	PortPinN2Het2_8	PORT_PIN_OUT	☒	8	PORT_PIN_MODE_PWM	PORT_PIN_LEVEL_LOW	POR
9	PortPinN2Het2_9	PORT_PIN_IN	☒	9	PORT_PIN_MODE_ICU	PORT_PIN_LEVEL_LOW	POR
10	PortPinN2Het2_10	PORT_PIN_IN	☒	10	PORT_PIN_MODE_ICU	PORT_PIN_LEVEL_LOW	POR
11	PortPinN2Het2_11	PORT_PIN_IN	☒	11	PORT_PIN_MODE_ICU	PORT_PIN_LEVEL_LOW	POR
12	PortPinN2Het2_12	PORT_PIN_OUT	☒	12	PORT_PIN_MODE_ICU	PORT_PIN_LEVEL_LOW	POR
13	PortPinN2Het2_13	PORT_PIN_IN	☒	13	PORT_PIN_MODE_ICU	PORT_PIN_LEVEL_LOW	POR
14	PortPinN2Het2_14	PORT_PIN_IN	☒	14	PORT_PIN_MODE_ICU	PORT_PIN_LEVEL_LOW	POR
15	PortPinN2Het2_15	PORT_PIN_OUT	☒	15	PORT_PIN_MODE_ICU	PORT_PIN_LEVEL_HIGH	POR
16	PortPinN2Het2_16	PORT_PIN_OUT	☒	16	PORT_PIN_MODE_ICU	PORT_PIN_LEVEL_HIGH	POR
17	PortPinN2Het2_18	PORT_PIN_OUT	☒	18	PORT_PIN_MODE_ICU	PORT_PIN_LEVEL_LOW	POR

The bottom of the interface shows the Error Log and Problems View, indicating no errors or warnings.

Pin Configuration

Configuration Table for Pins

```

portPin_cfg_t const PORT_config[PORT_NO_OF_PINS] =
{
    //Port 00
    #if (ASCLIN3 == UART6)
        {(Ifx_P *)&MODULE_P00, 0, _outputPushPullAlt2, PES_inactive, _cleared, cmosAutomotiveSpeed1}, //Port 00 Pin 0
    #else
        {(Ifx_P *)&MODULE_P00, 0, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 0
    #endif
    #if (ASCLIN3 == UART6)
        {(Ifx_P *)&MODULE_P00, 1, _inputPullUp, PES_inactive, _cleared, cmosAutomotiveSpeed1}, //Port 00 Pin 0
    #else
        {(Ifx_P *)&MODULE_P00, 1, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 1
    #endif

    {(Ifx_P *)&MODULE_P00, 2, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 2
    {(Ifx_P *)&MODULE_P00, 3, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 3
    {(Ifx_P *)&MODULE_P00, 4, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 4
    {(Ifx_P *)&MODULE_P00, 5, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 5
    {(Ifx_P *)&MODULE_P00, 6, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 6
    {(Ifx_P *)&MODULE_P00, 7, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 7
    {(Ifx_P *)&MODULE_P00, 8, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 8
    {(Ifx_P *)&MODULE_P00, 9, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 9
    {(Ifx_P *)&MODULE_P00, 10, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 10
    {(Ifx_P *)&MODULE_P00, 11, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 11
    {(Ifx_P *)&MODULE_P00, 12, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 12
    {(Ifx_P *)&MODULE_P00, 13, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 13
    {(Ifx_P *)&MODULE_P00, 14, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 14
    {(Ifx_P *)&MODULE_P00, 15, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 00 Pin 15

    //Port 01
    {(Ifx_P *)&MODULE_P01, 0, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 0
    {(Ifx_P *)&MODULE_P01, 1, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 1
    {(Ifx_P *)&MODULE_P01, 2, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 2
    {(Ifx_P *)&MODULE_P01, 3, _outputPushPullAlt5, PES_inactive, _cleared, cmosAutomotiveSpeed2}, //Port 01 Pin 3
    {(Ifx_P *)&MODULE_P01, 4, _inputPullUp, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 4
    {(Ifx_P *)&MODULE_P01, 5, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 5
    {(Ifx_P *)&MODULE_P01, 6, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 6
    {(Ifx_P *)&MODULE_P01, 7, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 7
    {(Ifx_P *)&MODULE_P01, 8, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 8
    {(Ifx_P *)&MODULE_P01, 9, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 9
    {(Ifx_P *)&MODULE_P01, 10, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 10
    {(Ifx_P *)&MODULE_P01, 11, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 11
    {(Ifx_P *)&MODULE_P01, 12, PORT_DEFAULT_MODE, PES_inactive, _cleared, cmosAutomotiveSpeed4}, //Port 01 Pin 12

```

Peripheral Configuration

Pin Definition

```
//ASCLIN 0 (USB)
{
    ON,                                     //ASCLIN 0 enabled
    115200,                                //ASCLIN 0 baud rate
    8,                                     //ASCLIN 0 data bits (7-9)
    UART_ParNone,                          //ASCLIN 0 parity
    UART_StopBit_1,                        //ASCLIN 0 stop bits
    OFF,                                   //ASCLIN 0 LoopBack
    RX_sel_a,                              //ASCLIN 0 RX pin input selector
    OFF,                                   //ASCLIN 0 CTS (Hardware flow control TX)
    UART_CTS_ActiveLOW,                     //ASCLIN 0 CTS polarity
    0,                                     //ASCLIN 0 CTS pin select (0-3)
    _ASCLIN_FIFO_MINLEVEL,                  //ASCLIN 0 TX Fifo interrupt fill level (top down, if the buffer empties)
    _ASCLIN_FIFO_MAXLEVEL,                  //ASCLIN 0 RX Fifo interrupt fill level (bottom up, if the buffer fills)
},

//ASCLIN 1
{
    ON,                                     //ASCLIN 1 enabled
    115200,                                //ASCLIN 1 baud rate
    8,                                     //ASCLIN 1 data bits (7-9)
    UART_ParNone,                          //ASCLIN 1 parity
    UART_StopBit_1,                        //ASCLIN 1 stop bits
    OFF,                                   //ASCLIN 1 LoopBack
    RX_sel_g,                              //ASCLIN 1 RX pin input selector
    OFF,                                   //ASCLIN 1 CTS (Hardware flow control TX)
    UART_CTS_ActiveLOW,                     //ASCLIN 1 CTS polarity
    0,                                     //ASCLIN 1 CTS pin select (0-3)
    _ASCLIN_FIFO_MINLEVEL,                  //ASCLIN 1 TX Fifo interrupt fill level (top down, if the buffer empties)
    _ASCLIN_FIFO_MAXLEVEL,                  //ASCLIN 1 RX Fifo interrupt fill level (bottom up, if the buffer fills)
},
```

Must be kept in
sync with overall
Pin table...

Peripheral API Design

Synchronous Drivers

Pin API

Function declarations:

- FL_QDA_Write(uint8) : void
 - value : uint8
- FL_QDA_SetDriveMode(uint8) : void
 - mode : uint8
- FL_QDA_ReadDataReg() : uint8
- FL_QDA_Read() : uint8
- FL_QDA_SetInterruptMode(uint16, uint16) : void
 - position : uint16
 - mode : uint16
- FL_QDA_ClearInterrupt() : uint8

PWM API

Function declarations:

- PWM_Rear_Start() : void
- PWM_Rear_Stop() : void
- PWM_Rear_SetInterruptMode(uint8) : void
 - interruptMode : uint8
- PWM_Rear_ReadStatusRegister() : uint8
- PWM_Rear_ReadControlRegister() : uint8
- PWM_Rear_WriteControlRegister(uint8) : void
 - control : uint8
- PWM_Rear_ReadCounter() : uint16
- PWM_Rear_ReadCapture() : uint16
- PWM_Rear_ClearFIFO() : void
- PWM_Rear_WriteCounter(uint16) : void
 - counter : uint16
- PWM_Rear_WritePeriod(uint16) : void
 - period : uint16
- PWM_Rear_ReadPeriod() : uint16
- PWM_Rear_WriteCompare1(uint16) : void
 - compare : uint16
- PWM_Rear_ReadCompare1() : uint16
- PWM_Rear_WriteCompare2(uint16) : void
 - compare : uint16
- PWM_Rear_ReadCompare2() : uint16
- PWM_Rear_Init() : void
- PWM_Rear_Enable() : void
- PWM_Rear_Sleep() : void
- PWM_Rear_Wakeup() : void
- PWM_Rear_SaveConfig() : void
- PWM_Rear_RestoreConfig() : void

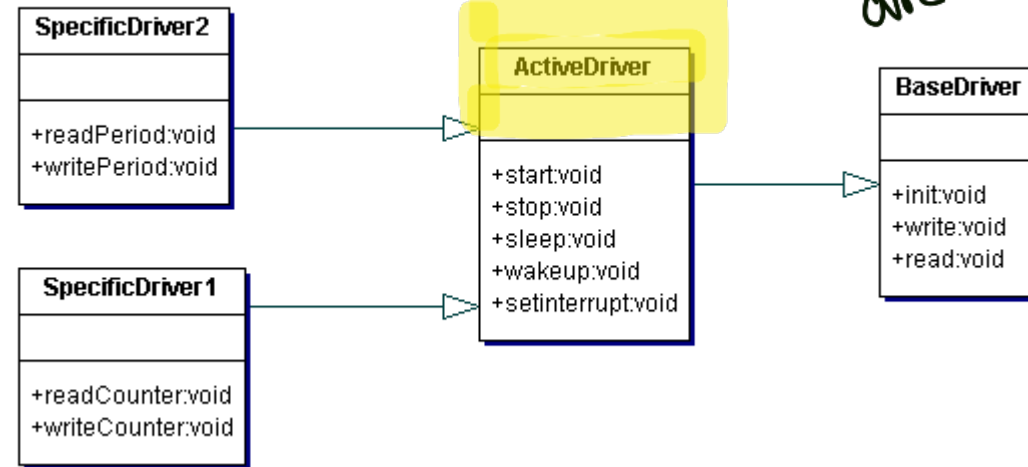
Q-Decoder API

Function declarations:

- RL_QuadDec_Init() : void
- RL_QuadDec_Start() : void
- RL_QuadDec_Stop() : void
- RL_QuadDec_Enable() : void
- RL_QuadDec_GetEvents() : uint8
- RL_QuadDec_SetInterruptMask(uint8) : void
 - mask : uint8
- RL_QuadDec_GetInterruptMask() : uint8
- RL_QuadDec_GetCounter() : int32
- RL_QuadDec_SetCounter(int32) : void
 - value : int32
- RL_QuadDec_Sleep() : void
- RL_QuadDec_Wakeup() : void
- RL_QuadDec_SaveConfig() : void
- RL_QuadDec_RestoreConfig() : void
- RL_QuadDec_ISR() : void

Driver API

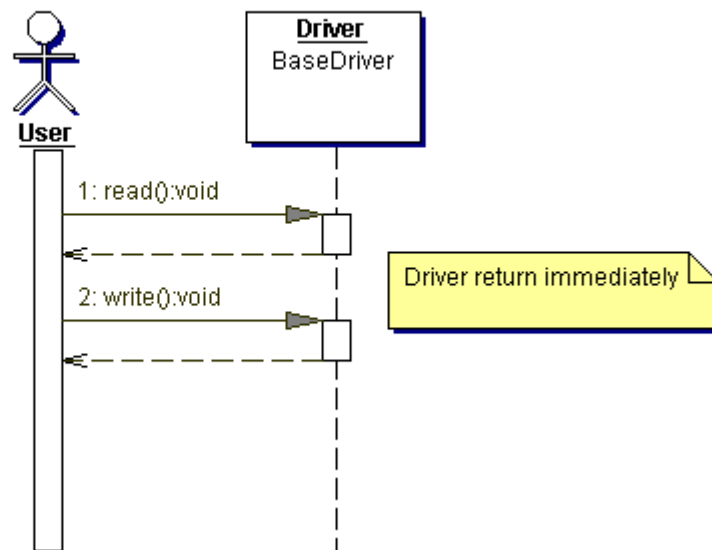
- Design Goal: An easy to understand and uniform API



→ A driver which is using a clock or some other dynamic elements, which are required to operate the driver is Active Driver

Synchronous Driver Implementation

- Synchronous Driver: User call waits for result of read() or write() function
- This implies that these functions should return immediately to avoid long blocking periods
- Typically this is true for many write functions, but read might have to wait for an external sender (e.g. USART, CAN,...)



```
int32 RL_QuadDec_GetCounter(void)
{
    int32 count;
    uint16 tmpCnt;

    #if (RL_QuadDec_COUNTER_SIZE == RL_QuadDec_COUNTER_SIZE_32_BIT)
        int16 hwCount;

        CyIntDisable(RL_QuadDec_ISR_NUMBER);

        tmpCnt = RL_QuadDec_Cnt16_ReadCounter();
        hwCount = (int16) ((int32) tmpCnt - (int32) RL_QuadDec_COUNTER_INIT_VALUE);
        count = RL_QuadDec_count32SoftPart + hwCount;

        CyIntEnable(RL_QuadDec_ISR_NUMBER);
    #else
        /* (RL_QuadDec_COUNTER_SIZE == RL_QuadDec_COUNTER_SIZE_8_BIT) ||
         * (RL_QuadDec_COUNTER_SIZE == RL_QuadDec_COUNTER_SIZE_16_BIT)
         */
        #if (RL_QuadDec_COUNTER_SIZE == RL_QuadDec_COUNTER_SIZE_8_BIT)
            tmpCnt = RL_QuadDec_Cnt8_ReadCounter();
        #else /* (RL_QuadDec_COUNTER_SIZE == RL_QuadDec_COUNTER_SIZE_16_BIT) */
            tmpCnt = RL_QuadDec_Cnt16_ReadCounter();
        #endif /* RL_QuadDec_COUNTER_SIZE == RL_QuadDec_COUNTER_SIZE_8_BIT */

        count = (int32) ((int32) tmpCnt -
            (int32) RL_QuadDec_COUNTER_INIT_VALUE);
    #endif /* RL_QuadDec_COUNTER_SIZE == RL_QuadDec_COUNTER_SIZE_32_BIT */

    return (count);
}
```

Single register read

Synchronous Drivers Implementation Rules

What you should do...

- Directly access the registers (read/write)
- Provide lots of comments

What you should avoid...

- (Big) Loops
- Complex logic (application or middleware)

What you NEVER EVER do

- Busy waiting
- Unterminated loops
- Non-volatile register definitions

Solution: Use a counter & set some defined value like 1000 and then terminate the while loop

→ Whenever we are calling a read or write function, it is valid for both, the operation will be terminated when the function is terminated.

→ The synchronous driver will block the system while it's being active.

→ pointers to registers have to be volatile.



```
void USART_sendByte(uint8_t data)
{
```

```
    //Wait until transmission register is empty
    while(REG_TX_STATUS != 0) {;}
```

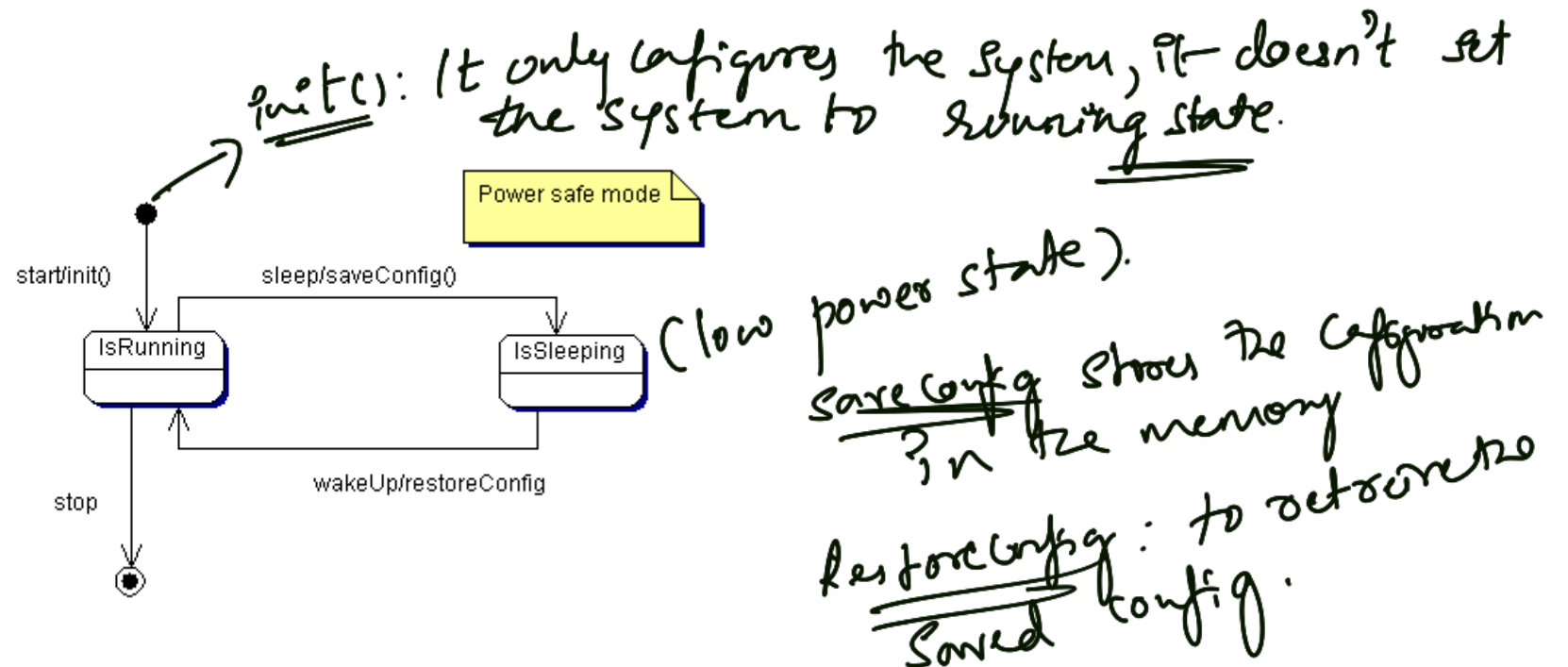
```
    //Write data into transmission register
    REG_TX_DATA = data;
}
```

} This is
Busy
Waiting

→ Watchdog & a brutal way of solving this Busy Waiting problem

Init versus Start/Stop versus Sleep/Wakeup

- There is no real standard for driver states, but at least some “conventions”
- Init is used to statically initialize the driver.
- Start / Stop are used to enter / leave the operational running state permanently
- Sleep/Wake up are used to support a low power state of the hardware, saving and restoring the driver state



Moving towards Asynchronous Drivers

Let's have a closer look at the USART driver...

- `UART_Zigbee_ReadRxData() : uint8`

```
/* Needs to check status for RX_STS_FIFO_NOTEMPTY bit */
rxData = UART_Zigbee_RXDATA_REG;
```

- `UART_Zigbee_PutString(const char8*) : void`

```
uint16 bufIndex = 0u;

/* If not Initialized then skip this function */
if(UART_Zigbee_initVar != 0u)
{
    /* This is a blocking function, it will not exit until all data is sent */
    while(string[bufIndex] != (char8) 0)
    {
        UART_Zigbee_PutChar((uint8) string[bufIndex]);
        bufIndex++;
    }
}
```

Function declarations:

- UART_Zigbee_Start() : void
- UART_Zigbee_Stop() : void
- UART_Zigbee_ReadControlRegister() : uint8
- UART_Zigbee_WriteControlRegister(uint8) : void
 - control : uint8
- UART_Zigbee_Init() : void
- UART_Zigbee_Enable() : void
- UART_Zigbee_SaveConfig() : void
- UART_Zigbee_RestoreConfig() : void
- UART_Zigbee_Sleep() : void
- UART_Zigbee_Wakeup() : void
- UART_Zigbee_SetRxAddressMode(uint8) : void
 - addressMode : uint8
- UART_Zigbee_SetRxAddress1(uint8) : void
 - address : uint8
- UART_Zigbee_SetRxAddress2(uint8) : void
 - address : uint8
- UART_Zigbee_SetRxInterruptMode(uint8) : void
 - intSrc : uint8
- UART_Zigbee_ReadRxData() : uint8
- UART_Zigbee_ReadRxStatus() : uint8
- UART_Zigbee_GetChar() : uint8
- UART_Zigbee_GetByte() : uint16
- UART_Zigbee_GetRxBufferSize() : uint8
- UART_Zigbee_ClearRxBuffer() : void
- UART_Zigbee_SetTxInterruptMode(uint8) : void
 - intSrc : uint8
- UART_Zigbee_WriteTxData(uint8) : void
 - txDataByte : uint8
- UART_Zigbee_ReadTxStatus() : uint8
- UART_Zigbee_PutChar(uint8) : void
 - txDataByte : uint8
- UART_Zigbee_PutString(const char8 *) : void
 - string : const char8 []
- UART_Zigbee_PutArray(const uint8 *, uint8) : void
 - string : const uint8 []
 - byteCount : uint8
- UART_Zigbee_PutCRLF(uint8) : void
 - txDataByte : uint8
- UART_Zigbee_ClearTxBuffer() : void
- UART_Zigbee_SetTxAddressMode(uint8) : void
 - addressMode : uint8
- UART_Zigbee_SendBreak(uint8) : void
 - retMode : uint8
- UART_Zigbee_GetTxBufferSize() : uint8

Oh my god !!!

Why did they do it??

Something we agreed never to do!!!!

```
while((UART_Zigbee_TXSTATUS_REG & UART_Zigbee_TX_STS_FIFO_FULL) != 0u)
{
    /* Wait for room in the FIFO */
}

/* Add directly to the FIFO */
UART_Zigbee_TXDATA_REG = txDataByte;
```

Key problem...

On the receiving side...

- We don't know, when the data will arrive. This is determined by the sender!
- A polling approach (using a slightly improved API with return value) would look like this:

```
while(1) {  
    uint8 data = 0;  
    if (true == USART_Zigbee_ReadRxData(&data)) {  
        //do something with the data  
        printf("Data Rx: %x\n", data);  
    }  
}
```

→ Here we are receiving the data in an asynchronous way.
→ This is not a blocking function.

Limitations

- Do nothing (== busy waiting) for most of the time
- We rely on the user not to ignore return values (and we still believe that Santa brings the presents)

Key problem...

On the sending side...

- We have to wait until the transmission register is available
- A polling approach would look like this:

```
char string[] = "Hello world";
uint16 i = 0;
while(1) {
    char data = string[i];
    if (0 == data) break; //End of string reached
    //Wait until transmission is possible
    while (USART_TXSTATUS_REG != 1){ ; }
    USART_TXDATA_REG = data
}
```

Limitations

- Storing the data byte in the register takes a few nanoseconds
- Waiting until transmission is available takes $n \times 10 \dots 100\text{ms}$ (way longer!!!!)

Improved asynchronous solution - Receiving

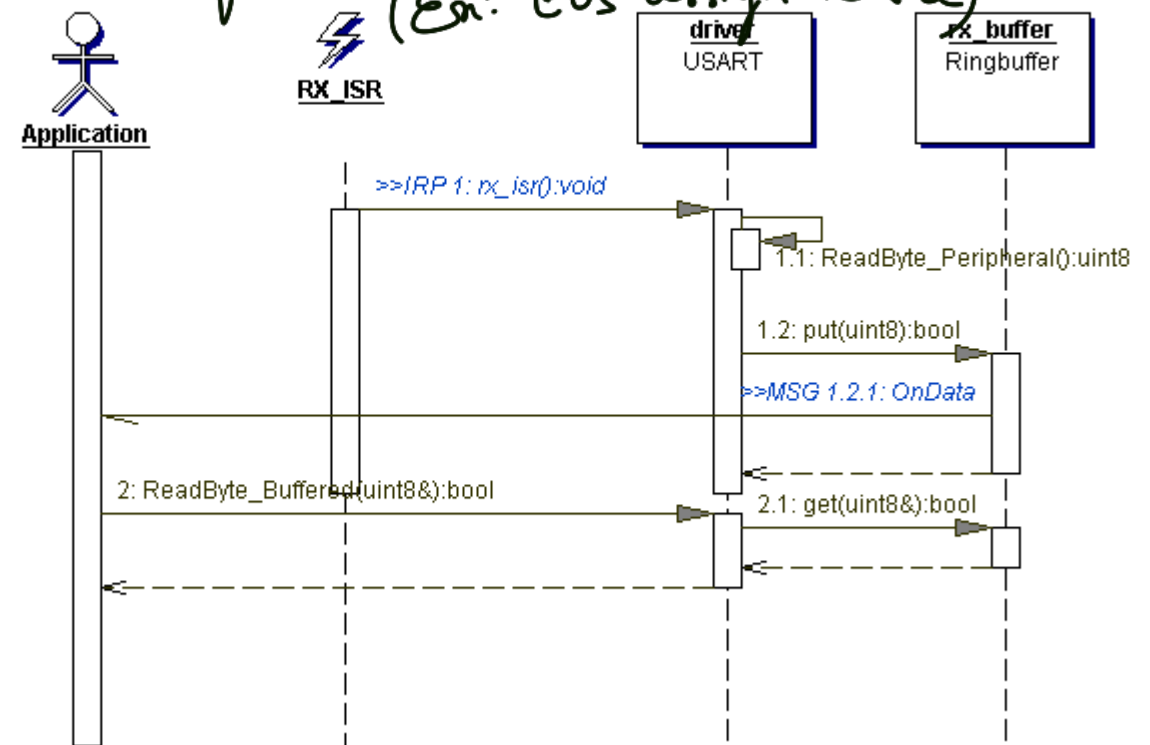
→ Whenever we receive a byte of data, an ISR is triggered automatically by the peripherals. The ISR then writes the data into the driver buffer. Once the buffer is full, the buffer will send out an (ev-protocol complete) event to the application. Then ASW will know that it has received a full block of data which can be further processed. (Ex: EOS assignment 2)

- A rx_isr is triggered whenever a byte is received by the hardware
- The rx_isr reads the data from the peripheral register
- And stores it in a shared buffer
- The application reads the data out of the buffer
- In case no data is present, an error code is returned

AddOn

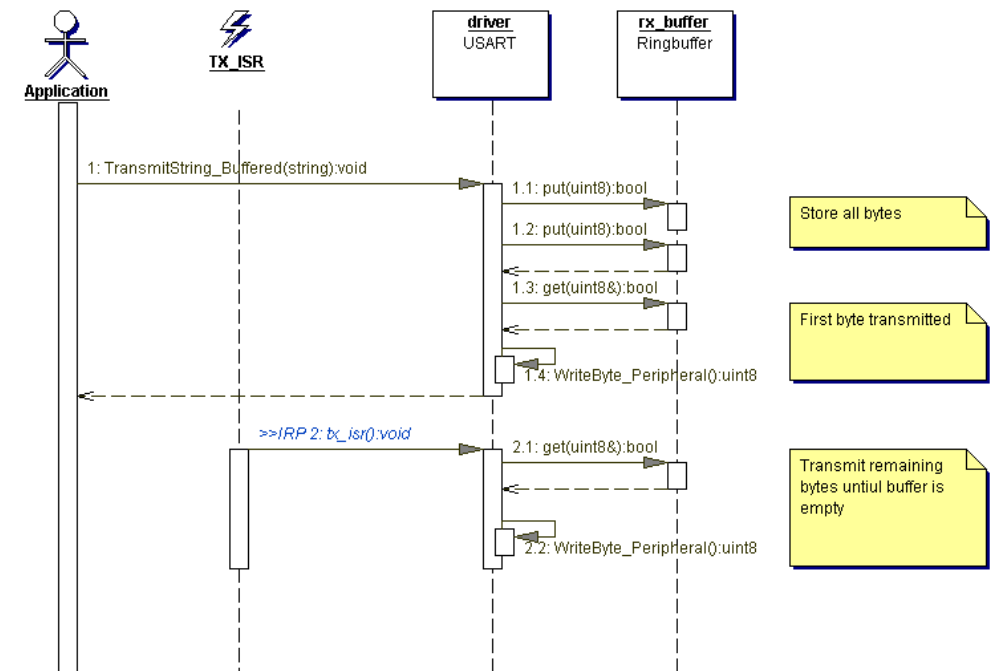
- The buffer sends an event to the applications if it needs to be read out.
- E.g. if buffer is full or protocol is complete

```
ISR2(ISR_UART_RX)
{
    uint8_t writedata;
    writedata = UART_ReadRxData();
    RingBuffer_Write(&RB_Obj, writedata);
    if (CO == writedata)
        setEvent(task_json, ev-Rxdata);
}
```



Improved asynchronous solution - Transmission

- The data to be transmitted is stored in a buffer
- A first byte is send in a synchronous way.
- Afterwards, the tx_isr takes over
- Reads the byte out of the buffer and sends it out as soon as the peripheral is ready.
- After the last byte has been read out, the transmit interrupt is cleared.
- Usually this happens automatically by not sending after the get() returned an empty buffer

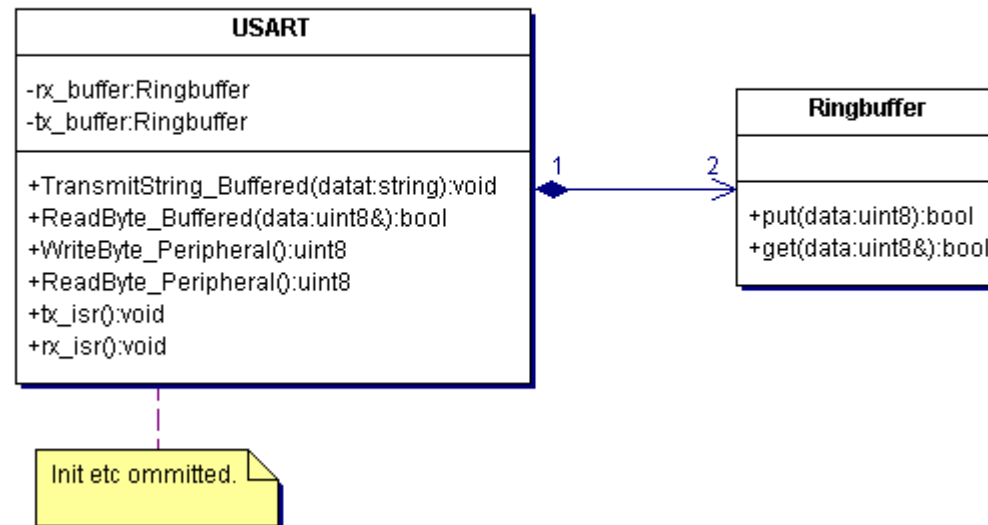


Resulting API

In addition to the synchronous API, we need

- Buffered read and write functions
- Buffers for the reception and transmission (FIFO ringbuffer)
- And interrupt services routines

→ Asynchronous drivers are more complex than synchronous drivers. The job is split into two parts, one job is for initiation & the other in background.



Asynchronous Drivers Implementation Rules

What you should do...

- Use buffered API's for ALL reception and LONG transmission services
- Add isr functions to the driver (which are then placed in the real isr hook)
- The isr should follow the pattern “get the data, store the data, get out”

What you should avoid...

- Complex logic (application or middleware) in the buffered transmission and reception functions
- Critical data sections in the buffer

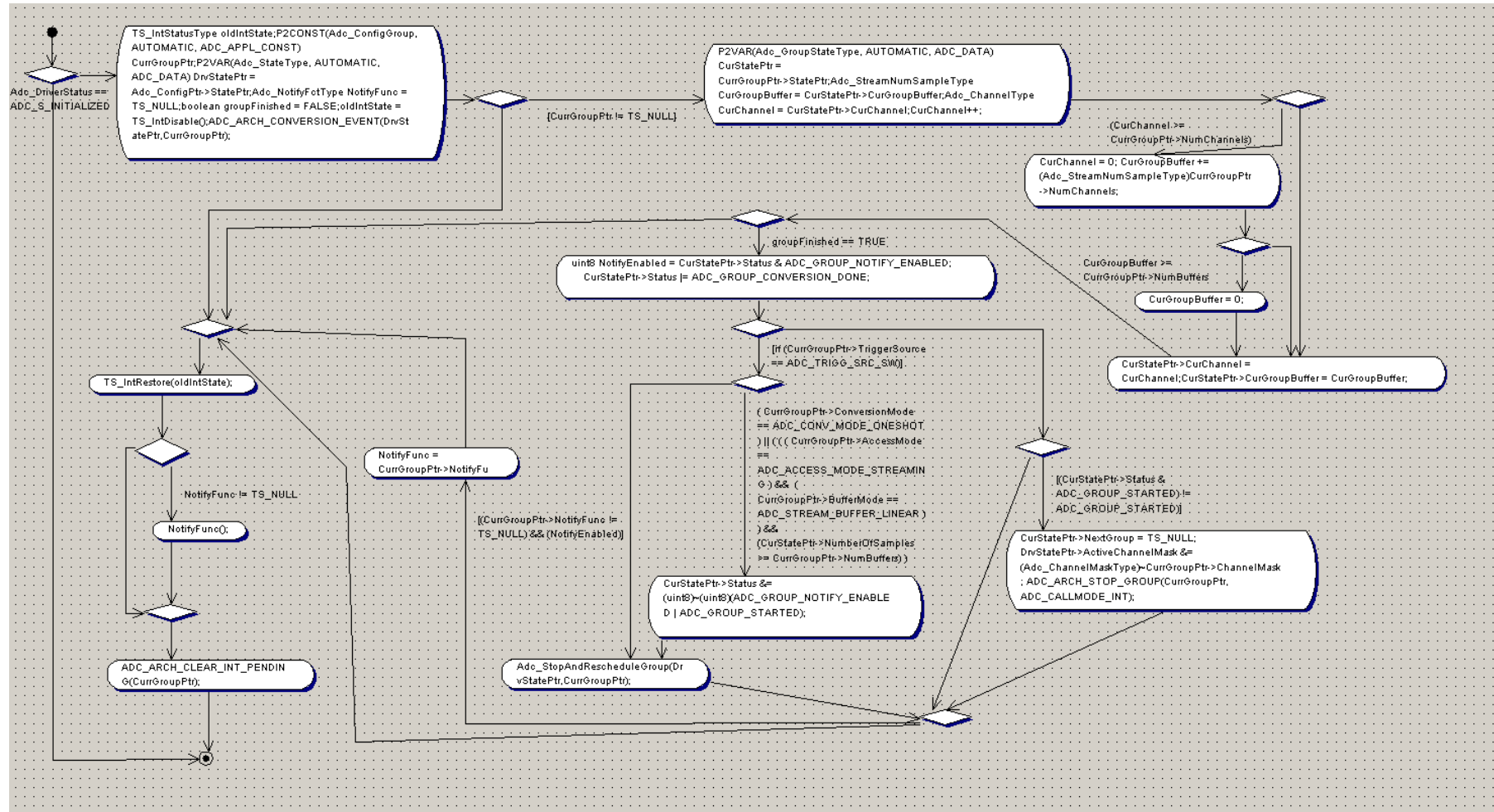
What you NEVER EVER do

- Loops in the isr
- Busy waiting in the isr



Interrupt Service Calls – Too Complex Design

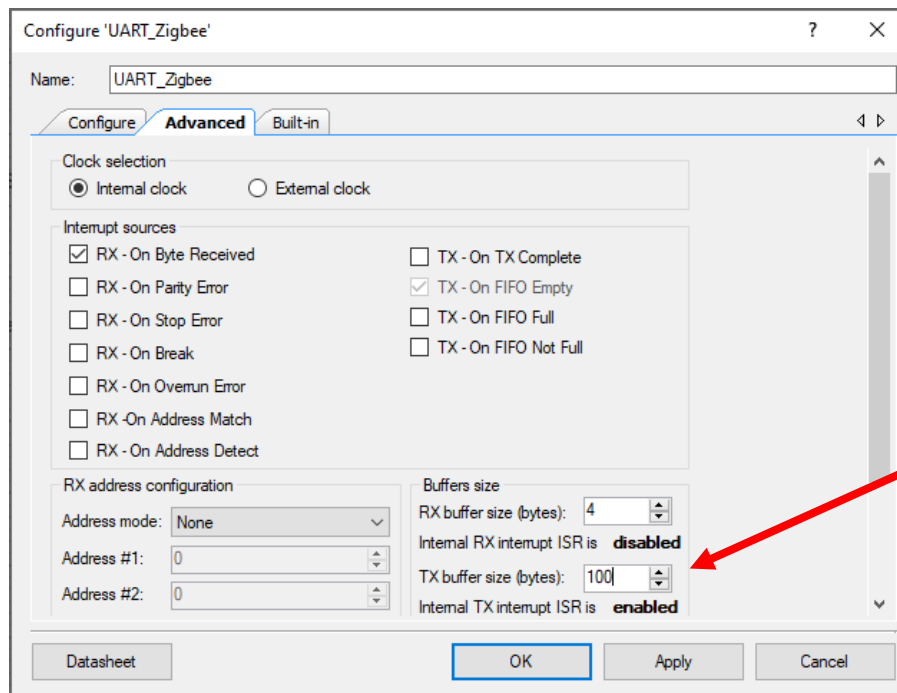
AUTOSAR ADC Interrupt realisation



Interrupt implementation alternatives

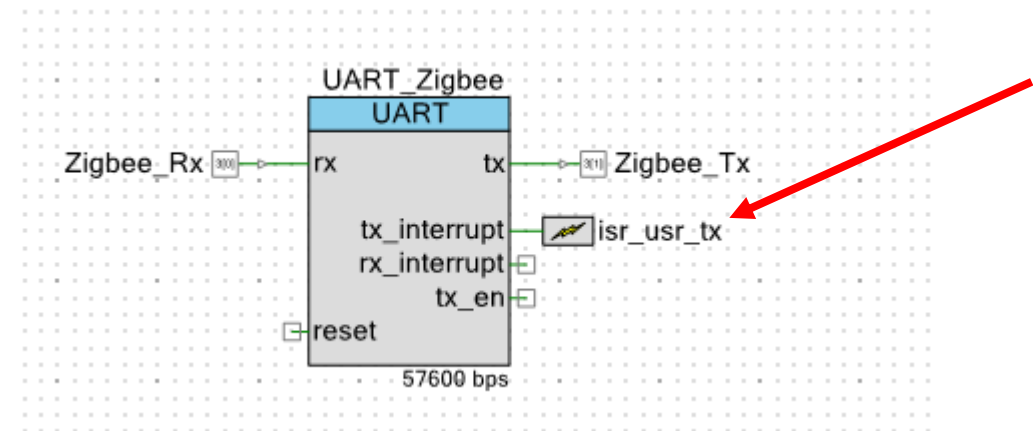
Option 1

- Internal interrupt function
- Good for simple buffered transmission



Option 2

- External interrupt
- More control, e.g. ev_OnProtocolComplete



Let's consider the following Use Case for an Asynchronous USART Driver

- Upon reception, the interrupt service routine shall store the received data in a ringbuffer
- If the last byte (EOP) has been received, the application shall be informed by sending a trigger event to the protocol handler task
- The most straight forward solution would be to add a SetEvent command to the driver
- But this would limit the applicability to a specific OS
- This is a huge problem for many driver libraries, e.g. Mbed, CMSIS,...



Critical sections

A similar problem are critical sections.

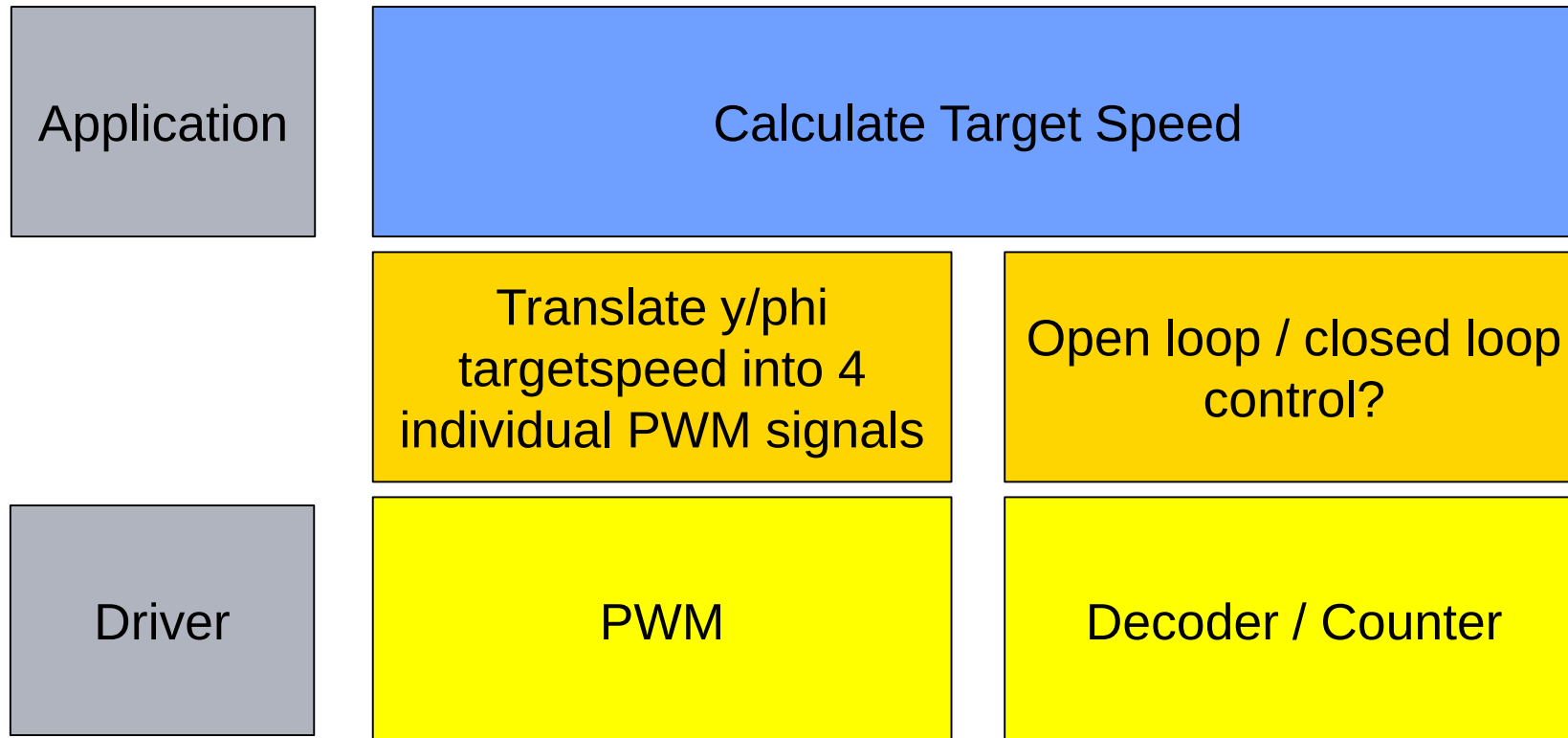
- Use mutex? ➔ OS specific, runtime penalty
- Disable all interrupts? ➔ Possible loss of data
- Avoid race by adding “assumption of use”, e.g.
 - the application will read the data only after receiving the trigger event.
 - Reading will be finished before the next interrupt comes in
 - ➔ Shifts responsibility to the application engineer.
- Use threadsafe buffers? ➔ Not so easy to be implemented, especially on multicores



Complex Device Driver (CDD)

What belongs to the driver, what must be part of the application?

Let's assume the following hierarchy for the car engine



Complex Device Driver

- Complex Device Driver implement complex sensor / actor functions.
- CDD's are typically not triggered by the RTE, instead they are directly called by Runnables
- CDD's typically call lower level MCSAL drivers

We will implement some CDD's in the upcoming lectures...

5 Introduction to CDD

A Complex Driver is a software entity not standardized by AUTOSAR that can access or be accessed via AUTOSAR Interfaces and/or Basic Software Modules APIs.

According to the document [3] *Layered Software Architecture*, a CDD is a specific module located in the Complex Drivers Layer of the Basic SoftWare which interacts with standard BSW modules or RTE.

- A CDD may need to interface to modules of the layered software architecture
- A module of the layered software architecture may need to interface to a CDD
- A CDD may need to interface SW-Cs via RTE

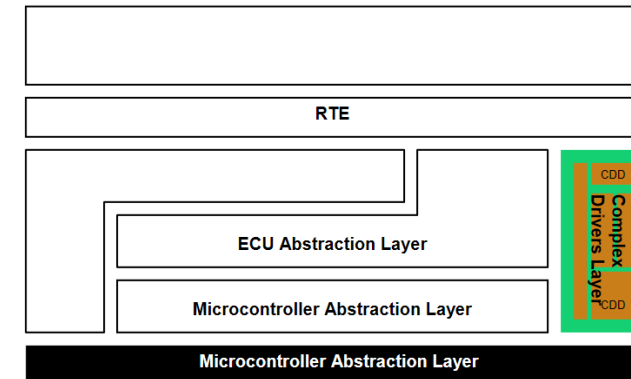


Figure 4.2-1: CDD in Layered Software Architecture

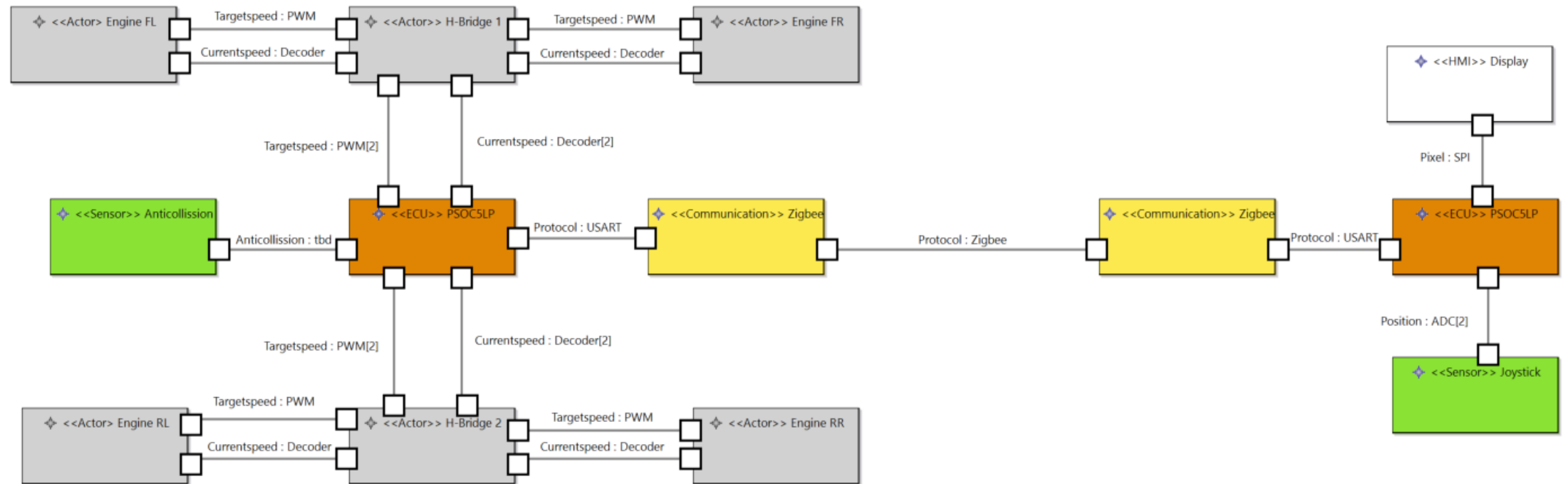
The main goal of the CDD is to implement complex sensor evaluation and actuator control with direct access to the microcontroller using specific interrupts and/or complex microcontroller peripherals, external devices (communication transceivers, ASIC...) to fulfill the special functional and timing requirements.

In addition it might be used to implement enhanced services / protocols or encapsulates legacy functionality of a non-AUTOSAR system.

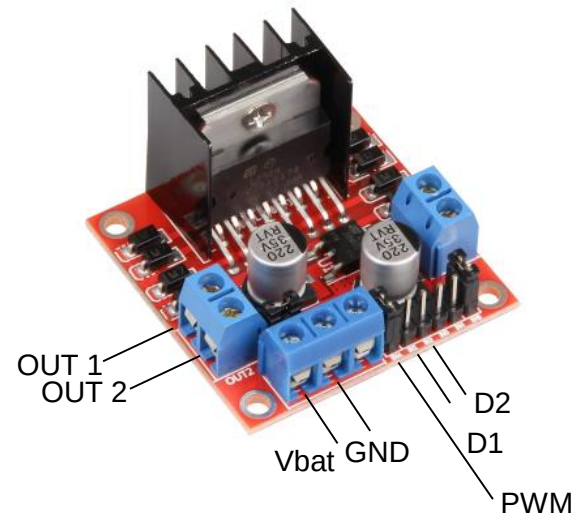
CDD Implementation might be application, μ C and ECU dependent.

Finally, the CDD can serve as mechanism of migration to introduce existing or new concepts into the AUTOSAR Software Architecture.

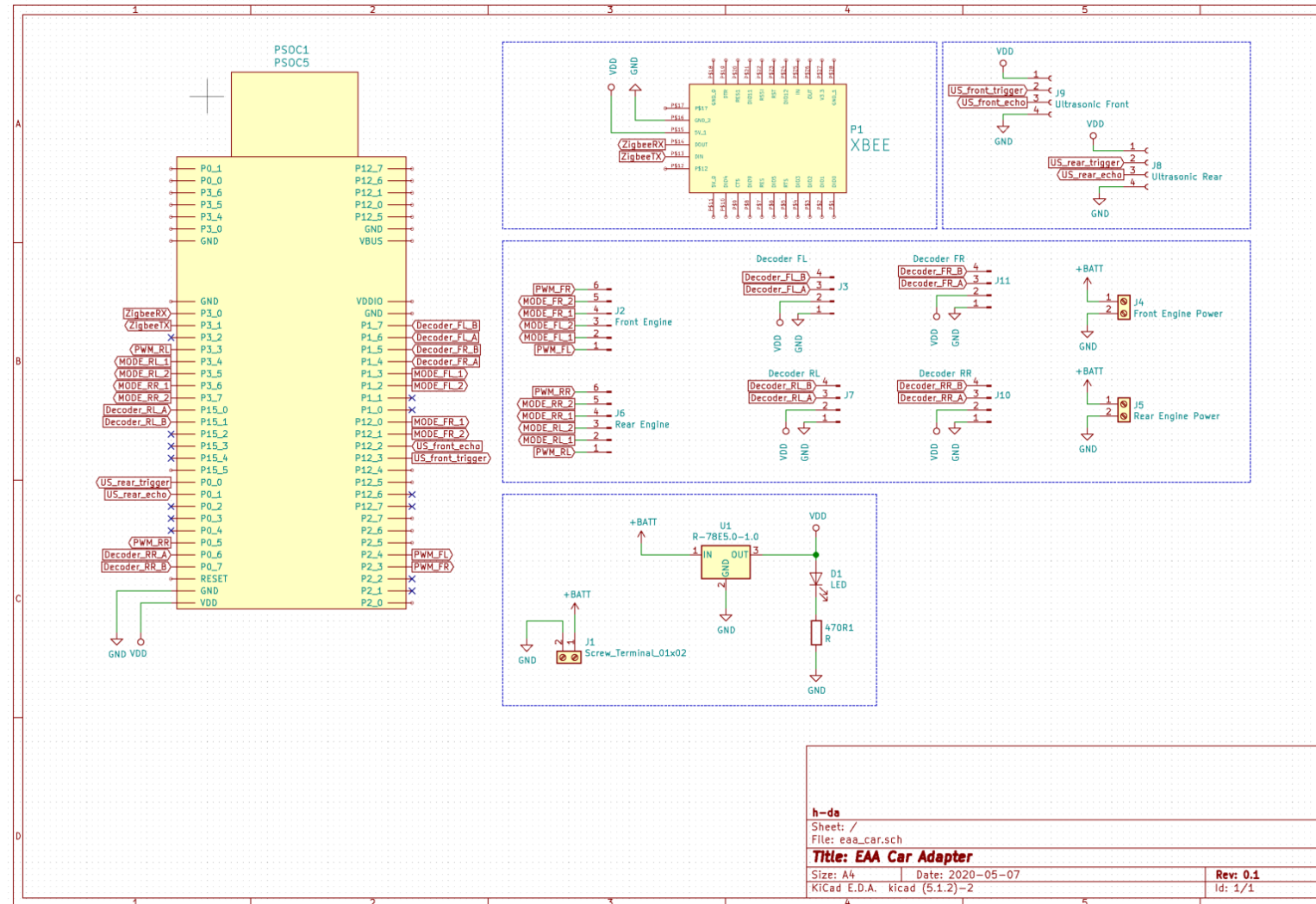
Let's create the MCAL for our car



Focus Engine and Decoder



Schematics



Wrapup

- Driver hierarchies
- Concepts for Pin and Peripheral configuration
- Synchronous driver design and implementation
- Asynchronous driver design and implementation
- Challenge: OS dependency